

5203 Project 3 Report

15-Class Scene Image Classification Based on Convolutional Neural Network (CNN)

April 22, 2026

1 Introduction

This report systematically describes the convolutional neural network (CNN) solution designed for the 15-class scene image classification task (15-Scene Categorization).

A lightweight VGG-style convolutional neural network, **SceneCNN**, was designed for this project, with targeted engineering optimisations introduced at every stage including data pre-processing, weight initialisation, regularisation, and learning rate scheduling. The final model achieved a best validation accuracy of **81.00%**.

2 Dataset and Pre-processing

2.1 Dataset Overview

The training dataset contains a total of 1,500 images spanning 15 scene categories, with exactly 100 images per category. The category list and the integer labels (1–15) specified by the project are shown in Table 1.

Table 1: 15-Class Scene Label Mapping (fixed as per project specification, must not be modified)

Label	Class	Label	Class	Label	Class
1	bedroom	6	Insidecity	11	OpenCountry
2	Coast	7	kitchen	12	store
3	Forest	8	livingroom	13	Street
4	Highway	9	Mountain	14	Suburb
5	industrial	10	Office	15	TallBuilding

2.2 Training and Validation Split

Using a fixed random seed (**seed** = 42), the 1,500 images are divided at an 8:2 ratio into:

- Training set: 1,200 images
- Validation set: 300 images

This splitting strategy ensures reproducibility, and the validation data leakage problem is completely avoided through a “dual-instantiation” strategy (see Section 4.2 for details).

Table 2: Comparison of transform pipelines for the training set and evaluation set

Training Set (Strong Augmentation)	Validation / Test Set (Deterministic Transforms)
RandomResizedCrop(224, scale=(0.2, 1.0))	Resize(256)
RandomHorizontalFlip(p=0.5)	CenterCrop(224)
ColorJitter(brightness, contrast, saturation=0.4)	—
ToTensor()	ToTensor()
Normalize(ImageNet mean & std)	Normalize(ImageNet mean & std)

2.3 Data Augmentation Strategy

Data augmentation is a key technique for improving model generalisation under limited data conditions. Separate transform pipelines are applied to the training set and the validation/test set, as shown in Table 2.

To prevent overfitting on small samples, `RandomResizedCrop` (`scale=(0.2, 1.0)`) serves as our core regularization, forcing the network to recognize scenes from partial crops (down to 20%) rather than relying solely on global context. Since Scene-15 is predominantly grayscale, images are converted to 3-channel RGB before applying `ColorJitter`. Here, `brightness` and `contrast` perturbations significantly enhance robustness against varying lighting conditions, while the `saturation` parameter—retained from the standard MAE pipeline—is harmlessly neutralized due to zero channel variance.

During evaluation, we adopt a deterministic `Resize(256)` and `CenterCrop(224)` paradigm to preserve aspect ratios and ensure input consistency. Finally, all images are normalized using ImageNet statistics. This entire preprocessing pipeline is adapted from Meta’s open-source MAE project (see Section 6 for details).

3 Network Architecture

3.1 Overall Design

The overall structure of `SceneCNN` consists of two parts: (1) the Feature Extractor backbone and (2) the MLP Classifier Head, as shown in Figure 1.

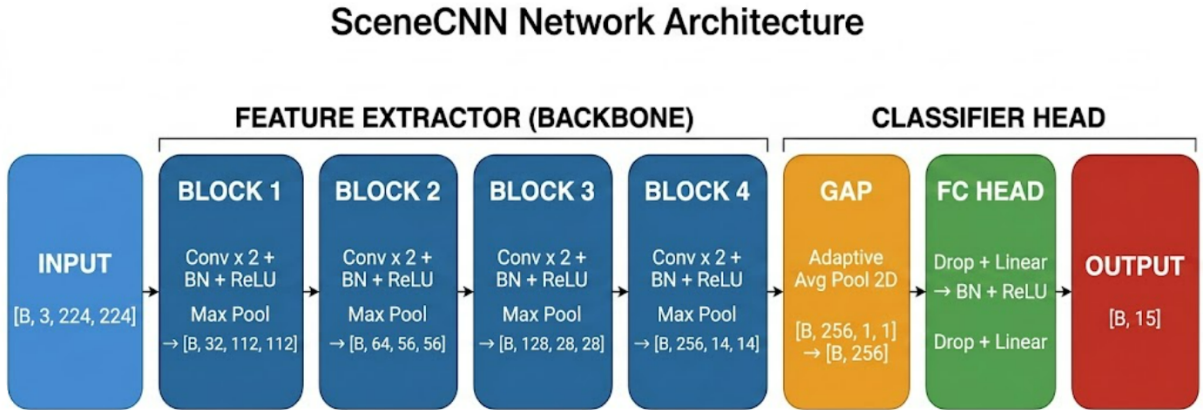


Figure 1: Schematic diagram of the SceneCNN network architecture

3.2 Double Convolution Blocks

The feature extractor backbone consists of 4 structurally identical Double Convolution Blocks connected in series. Each block has the following structure:

$$[\text{Conv}_{3 \times 3} \rightarrow \text{BN} \rightarrow \text{ReLU}] \times 2 \rightarrow \text{MaxPool}_{2 \times 2}$$

Two consecutive 3×3 convolutions achieve a receptive field equivalent to 5×5 with fewer parameters, while introducing more non-linear layers to enhance feature representation capacity. As the network depth increases, the channel capacity doubles in a stepwise manner: $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, while the spatial resolution is halved at each stage. Table 3 presents the dimensional changes at each module.

Table 3: Output dimensions of each layer in SceneCNN

Module	Input Dimension	Output Dimension	Notes
Block 1	$[B, 3, 224, 224]$	$[B, 32, 112, 112]$	in_c=3, out_c=32
Block 2	$[B, 32, 112, 112]$	$[B, 64, 56, 56]$	in_c=32, out_c=64
Block 3	$[B, 64, 56, 56]$	$[B, 128, 28, 28]$	in_c=64, out_c=128
Block 4	$[B, 128, 28, 28]$	$[B, 256, 14, 14]$	in_c=128, out_c=256
GAP	$[B, 256, 14, 14]$	$[B, 256, 1, 1]$	AdaptiveAvgPool2d(1,1)
Flatten	$[B, 256, 1, 1]$	$[B, 256]$	—
FC Head	$[B, 256]$	$[B, 15]$	Dropout + Linear + BN + ReLU + Dropout + Lin

3.3 Global Average Pooling (GAP)

Before the feature maps enter the classifier head, `nn.AdaptiveAvgPool2d((1,1))` is used to compress the spatial dimensions to 1×1 . Compared to the conventional Flatten operation (which would produce an input vector of $256 \times 14 \times 14 = 50,176$ dimensions), GAP fixes the input to the fully-connected layer at 256 dimensions, substantially reducing the parameter count. It also acts as an embedded structural regularisation mechanism, effectively suppressing overfitting under the small-sample setting.

3.4 MLP Classifier Head

The classifier head has the following structure:

$$\text{Dropout}(0.5) \rightarrow \text{Linear}(256, 256) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(256, 15)$$

The two Dropout layers ($p = 0.5$) force the network to randomly mask 50% of neurons during each forward pass, preventing the network from memorising training samples and thereby significantly improving generalisation ability.

4 Implementation Details and Engineering Techniques

4.1 Label Offset Mechanism (The ± 1 Trick)

PyTorch’s `CrossEntropyLoss` strictly requires target class indices to fall within the range $[0, C - 1]$, whereas the project specification requires scene labels to be 1 to 15. To this end, the following bidirectional mapping mechanism is implemented:

- **Training / Validation phase:** The read labels are decremented by 1 ($1-15 \rightarrow 0-14$) before being passed to the loss function.

- **Testing phase:** The predicted indices output by the network are incremented by 1 (0–14 $\rightarrow$ 1–15) before being compared against the original ground-truth labels.

This mechanism ensures full compatibility between the model and the evaluation script, without requiring any modification to the original label storage format of the dataset.

4.2 Prevention of Validation Data Leakage

In PyTorch, when a single `Dataset` instance is split using `random_split`, all subsets share the same underlying data object, causing the validation set to be incorrectly subjected to the training augmentation transforms. This project completely resolves the problem through the following “dual-instantiation” scheme:

1. Two independent `Scene15Dataset` objects are instantiated from the same data directory, bound to the training transforms and evaluation transforms respectively.
2. Exactly the same random generator (`torch.Generator().manual_seed(42)`) is passed to both `random_split` calls, ensuring consistent index partitioning.
3. The first subset is taken from the training instance and the second subset from the evaluation instance, thereby completely isolating the validation set at both the logical and physical levels, guaranteeing it is free from contamination by random crops or flips.

4.3 Kaiming Weight Initialisation

The default initialisation for convolutional layers in PyTorch is based on the Leaky ReLU assumption. Since this network employs standard ReLU activations (which mask approximately 50% of the variance energy in each forward pass), Kaiming Normal initialisation is explicitly applied:

```
1 nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
```

This initialisation precisely compensates for the variance attenuation caused by ReLU, effectively preventing the dead neuron problem in deep networks, and significantly accelerates convergence in the early stages of training.

5 Training Configuration and Results

5.1 Hyperparameter Settings

Table 4: Summary of training hyperparameters

Hyperparameter	Value
Optimiser	AdamW
Initial Learning Rate	1×10^{-3}
Weight Decay	1×10^{-4}
LR Scheduler	CosineAnnealingLR ($T_{\max} = 80$)
Training Epochs	80
Batch Size	32
Dropout Probability	0.5 (applied twice)
Input Image Size	224×224

AdamW implements weight decay more rigorously than standard Adam by decoupling L2 regularisation from the adaptive gradient term, providing superior generalisation on small-data tasks. The cosine annealing scheduler smoothly decays the learning rate from its initial value towards zero, facilitating fine convergence to a local optimum in the later stages of training.

5.2 Model Saving Strategy

The training script monitors the validation accuracy at every epoch in real time, and overwrites the model’s `state_dict` to the specified path only when a new best accuracy is achieved (best-checkpoint strategy). This automatically captures the weight snapshot with the strongest generalisation ability throughout the entire training process.

5.3 Training Curve

Figure 2 shows the trends of training loss and validation accuracy across epochs.

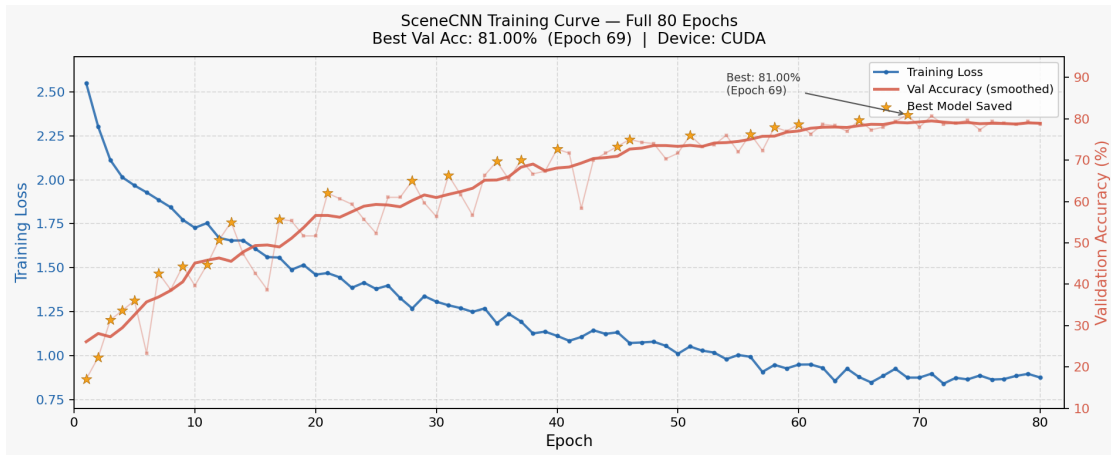


Figure 2: SceneCNN training curve (Epochs 1–80). The solid blue line is the training loss (left axis), the dashed red line is the validation accuracy (right axis), and orange stars mark the best model save points.

5.4 Final Results

Table 5: Summary of model performance

Metric	Value
Training Epochs	80
Final Training Loss (Epoch 80)	0.9522
Best Validation Accuracy	81.00%

6 Acknowledgements, Third-Party Code, and AI Usage Declaration

In accordance with the course guidelines on academic integrity, the following explicit declaration is made regarding the use of third-party open-source resources and generative artificial intelligence (Generative AI) tools in this project:

1. Third-Party Code and Resource Declaration

The data augmentation pipeline design adopted in the `build_transforms` function in `dataset.py`—in particular the pre-processing strategy combining `RandomResizedCrop` (`scale=(0.2, 1.0)`) with ImageNet normalisation parameters, as well as the evaluation-stage approach of `Resize`

followed by CenterCrop—is referenced and adapted from the MAE (Masked Autoencoders) project open-sourced on GitHub by Meta Platforms, Inc. This portion of the pre-processing logic is not the original independent work of the author, and is hereby acknowledged accordingly.

With the exception of the aforementioned data augmentation strategy, all core network architectures (`model.py`) and the complete training and inference framework (`scene_recog_cnn.py`) used in this project were independently designed and implemented by the author.

2. Generative AI Tool Usage Declaration

During the development and documentation of this project, large language models were used strictly as auxiliary productivity tools. The specific scope and boundaries of their use are as follows:

- **Coding & Debugging:** AI assistance was limited to debugging runtime errors and looking up official API documentation and standard usage examples for specific libraries such as PyTorch. All architectural design decisions, data flow logic, and hyperparameter choices were independently determined by the author.
- **Documentation:** AI tools were used to assist with structural formatting, textual polishing, grammar checking, and partial Chinese-English translation of this technical report and the project `README.md`. All core motivational discussions, experimental observations, and conclusions in the report originate from the author’s own experimental data and independent analysis.

7 Conclusion

This report provides a complete description of the design and implementation of a CNN-based 15-class scene classification system. Competitive classification performance under strict small-sample constraints was achieved through the following key engineering measures:

- Lightweight double-convolution backbone with Global Average Pooling to reduce overfitting risk;
- Two-layer Dropout regularisation;
- Kaiming weight initialisation to accelerate convergence;
- Dual-instantiation strategy to completely prevent validation data leakage;
- Label ± 1 bidirectional mapping to ensure compatibility with the evaluation script;
- AdamW combined with cosine annealing scheduling to optimise training stability;
- Best-checkpoint saving to automatically record the weight snapshot with the strongest generalisation ability.

References

- [1] Simonyan, K., & Zisserman, A. (2014). *Very deep convolutional networks for large-scale image recognition*. arXiv preprint arXiv:1409.1556.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving deep into rectifiers: Surpassing human-level performance on imagenet classification*. In Proceedings of the IEEE international conference on computer vision (pp. 1026–1034).
- [3] Loshchilov, I., & Hutter, F. (2016). *SGDR: Stochastic gradient descent with warm restarts*. arXiv preprint arXiv:1608.03983. (Used for Cosine Annealing Learning Rate scheduling).